Homework: Neural Population Dynamics, jPCA, and RNN Models of Motor Cortex

Neurobiological Basis of Intelligence

Lecture 2 Homework

Neural Code Representations & Network Dynamics

Due: June 1st Total: 100 points

Overview

This homework has three parts. In Part 1, you will replicate the jPCA analysis from Churchland et al. (2012) on publicly available motor cortex data and verify that rotational dynamics emerge in the state space. In Part 2, you will critically evaluate *why* these rotational dynamics appear—examining the quasi-oscillatory hypothesis from the original paper alongside alternative explanations. In Part 3, you will train a recurrent neural network (RNN) following the approach of Sussillo et al. (2015) and compare its learned dynamics to those observed in the real neural data.

Required Reading

1. Churchland, M.M., Cunningham, J.P., Kaufman, M.T., et al. (2012). Neural population dynamics during reaching. *Nature*, 487, 51–56.
2. Sussillo, D., Churchland, M.M., Kaufman, M.T., & Shenoy, K.V. (2015). A neural network that finds a naturalistic solution for the production of muscle activity. *Nature Neuroscience*, 18, 1025–1033.

Supplementary Reading (for Part 2)

1. Lebedev, M.A., Ninenko, I., Ossadtchi, A. (2019). Analysis of neuronal ensemble activity reveals the pitfalls and shortcomings of rotation dynamics. *Scientific Reports*, 9, 18978.
2. Lara, A.H., Cunningham, J.P., & Churchland, M.M. (2018). Different population dynamics in the supplementary motor area and motor cortex during reaching. *Nature Communications*, 9, 2754.
3. Euzmina,E.,Kriukov,D.,&Lebedev,M. (2024). Neuronal travelling waves explain rotational dynamics in experimental datasets and modelling. *Scientific Reports*, 14, 3566.

Supplementary Reading (for Part 3)

1. Clark, D.G., Bordelon, B., Zavatone-Veth, J.A., & Pehlevan, C. (2026). Structure, disorder, and dynamics in task-trained recurrent neural circuits. *bioRxiv*, 2026.03.02.708943. (*Provides the publicly available Sussillo data and reference RNN training code.*)

Data

This homework uses two data sources. For Parts 1 and 2, you may use either dataset below. For Part 3, you **must** use the Sussillo dataset, which includes EMG recordings.

Dataset A: MC_Maze (Neural Latents Benchmark). Recordings from primary motor cortex (M1) and dorsal premotor cortex (PMd) during a delayed center-out reaching task with barriers. Includes neural spiking data, hand position, and velocity, but *no EMG*.

- **DANDI Archive:** https://dandiarchive.org/dandiset/000128
- **NLB Tools (Python):** https://github.com/neurallatents/nlb_tools

Dataset B: Sussillo et al. (2015) data (recommended). The complete dataset from Sussillo et al. (2015), made publicly available by Clark, Bordelon, Zavatone-Veth, & Pehlevan (2026) with permission from Mark Churchland. This dataset contains trial-averaged firing rates from M1/PMd, **EMG targets from multiple arm muscles**, hand trajectories, and the RNN training inputs used in the original paper. The data is packaged as a single `sussillo_data.npz` file.

- **GitHub:** https://github.com/davidclark1/RNN-Learning-Theory
- **Zenodo (direct download):** Zenodo record (download `sussillo_data.tar`)
- **Data loader:** `sussillo_data/load_sussillo_data.py` converts raw MATLAB files to `.npz`
- **Data documentation:** `sussillo_data/README.md` for variable descriptions and provenance

The repository also includes a reference RNN training script (`monkey/train.py`) and analysis notebook (`monkey/analysis.ipynb`) from Clark et al. (2026). You may consult these for guidance, but you should write your own implementation for the homework.

Alternatively, the original jPCA MATLAB code with limited example data is available from: https://churchland.zuckermaninstitute.columbia.edu/content/code

Part 1: Replicating jPCA and Rotational Dynamics (40 points)

1.1 Data Preprocessing (10 pts)

Load the MC_Maze dataset. For each trial, extract the spike counts in the movement epoch (from movement onset to 300 ms after onset). Bin spikes into 10 ms windows and compute trial-averaged firing rates for each reach condition. Apply Gaussian smoothing ($\sigma = 20$ ms) to the trial-averaged rates.

Deliverables:

(a) Report the number of neurons, number of conditions, and the time window used. Note which neurons are from M1 vs. PMd (the leading digit of the unit ID indicates this: 1 = PMd, 2 = M1).

(b) Plot the trial-averaged firing rates for 6 example neurons (3 from M1, 3 from PMd) across all conditions. Comment on the diversity and temporal complexity of the responses. Do individual neurons exhibit simple tuning curves, or are the temporal profiles more complex?

1.2 PCA and Dimensionality Reduction (10 pts)

Apply PCA to the condition-averaged firing rate matrix (time × neurons, concatenated across conditions).

Deliverables:

(a) Plot the cumulative variance explained as a function of the number of principal components. How many PCs are needed to capture 80% and 90% of the variance?

(b) Project the neural trajectories onto the top 2 and top 3 PCs. Plot the trajectories for all conditions, color-coded by reach direction. Do you observe any rotational structure in these projections?

(c) Why is standard PCA alone insufficient for rigorously identifying rotational dynamics? (*Hint:* PCA finds axes of maximal variance, not axes of maximal rotation.)

1.3 Implementing jPCA (20 pts)

Implement the jPCA algorithm from scratch (do not use the original MATLAB toolbox directly, though you may use it to verify your results). The procedure is:

1. Reduce the population activity to the top 6 principal components, yielding a state vector $\mathbf{x}(t) \in \mathbb{R}^6$ for each condition at each time step.
2. Compute the numerical time derivative:

$$\dot{\mathbf{x}}(t) \approx \frac{\mathbf{x}(t+\Delta t) - \mathbf{x}(t-\Delta t)}{2\Delta t}$$

3. Pool all time points across all conditions and fit a linear dynamical system $\dot{\mathbf{x}} = M\mathbf{x}$ by least-squares regression.
4. Decompose M into its symmetric and skew-symmetric parts:

$$M_{\text{skew}} = \frac{M - M^\top}{2}, \qquad M_{\text{sym}} = \frac{M + M^\top}{2}$$

5. Compute the eigenvalues and eigenvectors of M_{skew}. The eigenvectors come in conjugate pairs defining 2D planes of maximal rotation (the jPCA planes).
6. Project the data onto the top jPCA plane (the pair with the largest-magnitude eigenvalues).

Deliverables:

(a) Report the fitted matrix M and its decomposition into M_{skew} and M_{sym}. Compute the ratio:

$$R^2 = \frac{\text{Var}(M_{\text{skew}}\,\mathbf{x})}{\text{Var}(M\,\mathbf{x})}$$

This ratio measures what fraction of the linear dynamics is rotational. What value do you obtain? Churchland et al. report values around 0.7–0.9.

(b) Report the eigenvalues of M_{skew}. Verify they are purely imaginary. What rotational frequencies (in Hz) do they correspond to, given your time bin size?

(c) Plot the neural trajectories projected onto the top two jPCA planes (jPC_1–jPC_2 and jPC_3–jPC_4). Use the same color scheme as in 1.2(b). Do all conditions rotate in the same direction? Is this consistent with the findings in the original paper?

(d) Compare your jPCA projections with the standard PCA projections from 1.2(b). What does the jPCA view reveal that PCA alone does not?

Part 2: Why Rotational Dynamics? (30 points)

2.1 The Quasi-Oscillatory Hypothesis (10 pts)

Churchland et al. argue that the rotational dynamics arise because individual neurons exhibit quasi-oscillatory firing patterns—multiphasic temporal profiles during movement that, when combined across the population, produce rotations in state space.

Deliverables:

(a) For each neuron, compute the number of peaks (local maxima) in the trial-averaged firing rate during the movement epoch, averaged across conditions. Plot a histogram of peak counts. What fraction of neurons have 2 or more peaks (i.e., are genuinely multiphasic)?

(b) For each neuron and condition, fit a damped sinusoidal model:

$$r(t) = A \cdot e^{-\alpha t} \cdot \cos(\omega t + \varphi) + b$$

Report the distribution of fitted frequencies ω across neurons and conditions. Is there a dominant frequency band? How does it compare to the rotational frequencies from your jPCA eigenvalues?

(c) In 2–3 paragraphs, summarize the quasi-oscillatory hypothesis. Why would a dynamical system that generates muscle commands *need* to produce oscillatory patterns? What is the link between the multiphasic structure of individual neurons and the population-level rotations?

2.2 Alternative Explanations (20 pts)

The rotational dynamics finding has generated significant debate. Several alternative explanations have been proposed for why jPCA finds rotational structure.

Explore at least two of the following alternatives and, for each, provide analysis and discussion:

(a) Sequential firing as a trivial source of rotations (10 pts)

If neurons fire in a temporal sequence (neuron 1, then neuron 2, ..., then neuron N), PCA on this activity will generically yield rotational trajectories. Lebedev et al. (2019) and others have argued that the jPCA rotations may simply reflect sequential activation patterns rather than a dynamical system.

- Construct a synthetic population of 50 neurons with sequential Gaussian activation peaks (staggered in time, no oscillatory structure within individual neurons). Apply your jPCA implementation. Do you find rotational dynamics? Compute the R^2 ratio.

- Compare the R^2 ratio, eigenvalue structure, and jPCA plane projections of your synthetic sequential data with the real motor cortex data. Can you distinguish genuine oscillatory dynamics from sequential activation using jPCA alone?
- Propose and implement a statistical test or control analysis that could distinguish between "rotational because of oscillations" and "rotational because of sequences." (*Hint:* consider the covariance-matched permutation test from Michaels et al. (2016), or compare the relationship between preparatory and movement-epoch activity.)

(b) Sensorimotor feedback (10 pts)

Rotational dynamics could arise not from autonomous cortical dynamics but from sensorimotor feedback loops. During reaching, proprioceptive and visual feedback returns to motor cortex with a delay, creating a loop: cortical command → muscle activation → limb movement → sensory feedback → cortical input. A delayed feedback loop around any dynamical system can produce oscillatory/rotational structure.

- Consider a simple model: $\dot{\mathbf{x}}(t) = A\mathbf{x}(t) + B\mathbf{x}(t-\tau)$, where τ is the feedback delay (∼50–100 ms). Under what conditions on A, B, and τ does this system exhibit rotational dynamics? (An analytical or numerical exploration is acceptable.)
- If rotational dynamics are feedback-driven, what predictions would you make about rotational structure in the *preparatory* period (before movement onset, when there is no movement and hence no feedback)? Check this against the data: apply jPCA to the preparatory epoch. Do you find rotations?
- Discuss: Sussillo et al. (2015) tested RNN variants with and without simulated sensory feedback. Both variants exhibited rotational dynamics. What does this tell us about the role of feedback?

(c) Traveling waves as an alternative geometric interpretation (10 pts)

Vaidya et al. (2024) proposed that the rotational dynamics revealed by jPCA are more parsimoniously explained as traveling waves of neural activity across the cortical surface, rather than oscillatory dynamics within individual neurons.

- Explain the traveling wave hypothesis in 2–3 paragraphs. How does a spatially traveling wave across a cortical sheet map onto rotations in PCA/jPCA space?
- If you have access to electrode position information in the dataset, test whether neurons with nearby spatial positions tend to have nearby peak times. Plot peak firing time vs. electrode position. Is there evidence of spatial ordering consistent with a traveling wave?
- Discuss: Are "traveling waves" and "rotational dynamics" truly alternative explanations, or are they the same phenomenon described at different levels of analysis?

Part 3: Training an RNN to Produce Muscle Activity (30 points)

Background: Sussillo et al. (2015)

Sussillo, Churchland, Kaufman, and Shenoy (2015) trained recurrent neural networks to reproduce the electromyographic (EMG) signals recorded from multiple arm muscles during the same reaching task. The key design choices were:

- **Architecture:** A continuous-time RNN with $N = 200$ recurrent units. The dynamics follow:

$$\tau\,\dot{\mathbf{r}} = -\mathbf{r} + f(W_{\text{rec}}\,\mathbf{r} + W_{\text{in}}\,\mathbf{u} + \mathbf{b})$$

  where $\mathbf{r}$ is the vector of firing rates, f is a pointwise nonlinearity (tanh), W_{rec} is the recurrent weight matrix, W_{in} maps inputs to the network, and $\mathbf{u}$ is the external input.

- **Inputs:** A condition-specific input signal (encoding reach direction) provided during the preparatory period and removed at movement onset, plus a go cue signal at movement onset.

- **Output:** A linear readout $W_{\text{out}}\,\mathbf{r}$ that maps firing rates to EMG signals for multiple muscles:

$$\mathbf{y} = W_{\text{out}}\,\mathbf{r} + \mathbf{b}_{\text{out}}$$

- **Training objective:** Minimize the mean squared error between the RNN output and the recorded EMG during the movement epoch. The RNN was *not* trained to match neural activity—only to produce the correct muscle outputs.

- **Regularization:** Critically, the RNN was regularized to encourage simple dynamics. Regularization terms penalized the magnitude of firing rates ($\|\mathbf{r}\|^2$), the input drive ($\|W_{\text{rec}}\,\mathbf{r}\|^2$), and the squared norm of the weight matrices. The strength of regularization was a key parameter: heavily regularized networks produced dynamics that closely resembled real motor cortex, while unregularized networks did not.

- **Training method:** Hessian-free optimization (Martens & Sutskever, 2011), though modern implementations can use BPTT with Adam.

The central finding was that when the RNN was strongly regularized, it discovered a low dimensional oscillatory solution—essentially a dynamical pattern generator—whose internal dynamics resembled those of real motor cortex both at the single-neuron level (complex, multiphasic firing patterns) and at the population level (rotational dynamics in jPCA). This was remarkable because the network was never shown any neural data; the resemblance emerged purely from the task constraint (produce the right EMGs) plus the simplicity bias (regularization).

3.1 Building the RNN (10 pts)

Implement a continuous-time RNN in PyTorch (or a framework of your choice) with the following specifications:

- $N = 200$ recurrent units with tanh nonlinearity
- Time constant $\tau = 50\,\text{ms}$
- Condition-specific input (2D vector encoding reach direction) provided during the preparatory period
- A binary go cue signal at movement onset

- Linear readout to K output channels, where K is the number of EMG channels in the dataset

EMG data. Use the EMG targets from the Sussillo et al. (2015) dataset (`sussillo_data.npz`), which contains trial-averaged rectified and smoothed EMG from multiple arm muscles recorded simultaneously with the neural data. These are the *same* EMG signals used as RNN training targets in the original paper. Load the data using the provided `load_sussillo_data.py` script or directly from the `.npz` file (see `sussillo_data/README.md` for field names and shapes). The EMG targets, the condition-specific inputs, and the time axis are all included.

Deliverables:

(a) Describe your network architecture, including the number and form of inputs, the recurrent dynamics equation, the readout, and how you handle the preparatory vs. movement epochs.

(b) Define your loss function. Include both the task loss (MSE on outputs during movement epoch) and regularization terms. Following Sussillo et al., add:

(i) L2 on firing rates: $\lambda_r \cdot \|\mathbf{r}\|^2$

(ii) L2 on recurrent input: $\lambda_u \cdot \|W_{\text{rec}}\,\mathbf{r}\|^2$

(iii) L2 on weights: $\lambda_w \cdot \left(\|W_{\text{rec}}\|^2 + \|W_{\text{in}}\|^2 + \|W_{\text{out}}\|^2\right)$

Report the values of λ_r, λ_u, λ_w you use.

3.2 Training and Evaluating the RNN (10 pts)

Train the RNN using backpropagation through time (BPTT) with Adam optimizer. Train until the network can accurately reproduce the target outputs for all reach conditions.

Deliverables:

(a) Plot the training loss over epochs. Demonstrate that the RNN has learned to produce reasonable outputs by plotting the target vs. predicted outputs for 4 representative conditions.

(b) Train two versions of the RNN: one with strong regularization and one with weak or no regularization. Report the final task loss and the regularization loss for both.

(c) Visualize single-unit activity in both networks for 6 example units across all conditions. In the strongly regularized network, do you observe multiphasic, temporally complex firing patterns similar to those in the real data (Part 1.1b)? How do the weakly regularized units differ?

3.3 Comparing RNN Dynamics to Neural Data (10 pts)

Apply your jPCA implementation from Part 1 to the hidden-unit activity of both the strongly and weakly regularized RNNs.

Deliverables:

(a) Compute PCA and jPCA on the RNN hidden-unit trajectories. Plot the jPCA projections for both networks alongside the real data projections from Part 1.3(c). Do the strongly regularized RNN dynamics resemble the real data?

(b) Compute the R^2 ratio (fraction of dynamics that is rotational) for both networks. Compare with the real data. Does stronger regularization produce more rotational dynamics?

(c) Compute canonical correlations (CCA) between the top-6 PC trajectories of the real neural data and the top-6 PC trajectories of each RNN. Following Sussillo et al., this provides a quantitative measure of how well the RNN dynamics match the neural dynamics. Report the top 3 canonical correlations for each network.

(d) In 3–4 paragraphs, discuss: Why does regularization push the RNN toward oscillatory, rotational dynamics? What does this suggest about why real motor cortex might exhibit similar dynamics? Relate your findings to the concept of Occam's razor in neural computation—the idea that biological circuits, constrained by metabolic costs and wiring economy, may favor the simplest dynamics capable of performing the task.

Submission Guidelines

Submit a Jupyter notebook (or equivalent) containing your code, figures, and written responses. All figures should be clearly labeled with axes, titles, and legends. Written responses should be concise but complete.

Grading: Part 1 (40 pts) + Part 2 (30 pts) + Part 3 (30 pts) = 100 pts total.

For Part 2, you must complete Section 2.1 (10 pts) and at least **two** of the three alternatives in Section 2.2 (10 pts each, 20 pts total).

Hints and Tips

- **Loading the Sussillo dataset:** Download `sussillo_data.tar` from Zenodo, extract it, and load `sussillo_data.npz` with `np.load()`. The file contains trial-averaged firing rates, EMG targets, hand trajectories, and input signals. See `sussillo_data/README.md` in the GitHub repo for the exact field names and array shapes.
- **For the MC_Maze dataset** (if used for Parts 1–2): `pynwb` and the `nlb_tools` package handle data loading. See the NLB tutorials at https://github.com/neurallatents/nlb_tools for starter code.
- When implementing jPCA, be careful about the numerical derivative. The central difference $\big(\mathbf{x}(t+\Delta t) - \mathbf{x}(t-\Delta t)\big)/(2\Delta t)$ is more accurate than the forward difference.
- For the RNN, start with a small network ($N = 50$) to debug, then scale up to $N = 200$.
- The regularization strength is the most important hyperparameter. Start with $\lambda_r = \lambda_u = 1.0$ and $\lambda_w = 0.1$, then adjust. The "right" regularization produces networks with moderate activity levels and smooth, low-dimensional dynamics.
- The Clark et al. (2026) repo includes a reference training script (`monkey/train.py`) that uses stochastic gradient Langevin dynamics (SGLD) rather than standard SGD/Adam. You may use either optimizer, but if you use Adam, note that the regularization strengths may need different tuning.
- For CCA, use `sklearn.cross_decomposition.CCA` or implement it via SVD on the cross-covariance matrix.